

A vertical bar on the left side of the slide with a gradient from orange at the top to blue at the bottom.

Unit 3

2.Multithreaded Programming

Multithreading

- Multithreading in Java is a process of executing multiple threads simultaneously.
- A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.
- However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Advantages of Java Multithreading

- 1) It doesn't block the user because threads are independent
- 2) You can perform many operations together, so it saves time.
- 3) Since Threads are independent, it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

Process-based Multitasking (Multiprocessing)

- › Each process has an address in memory. In other words, each process allocates a separate memory area.
- › A process is heavyweight.
- › Cost of communication between the process is high.
- › Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.

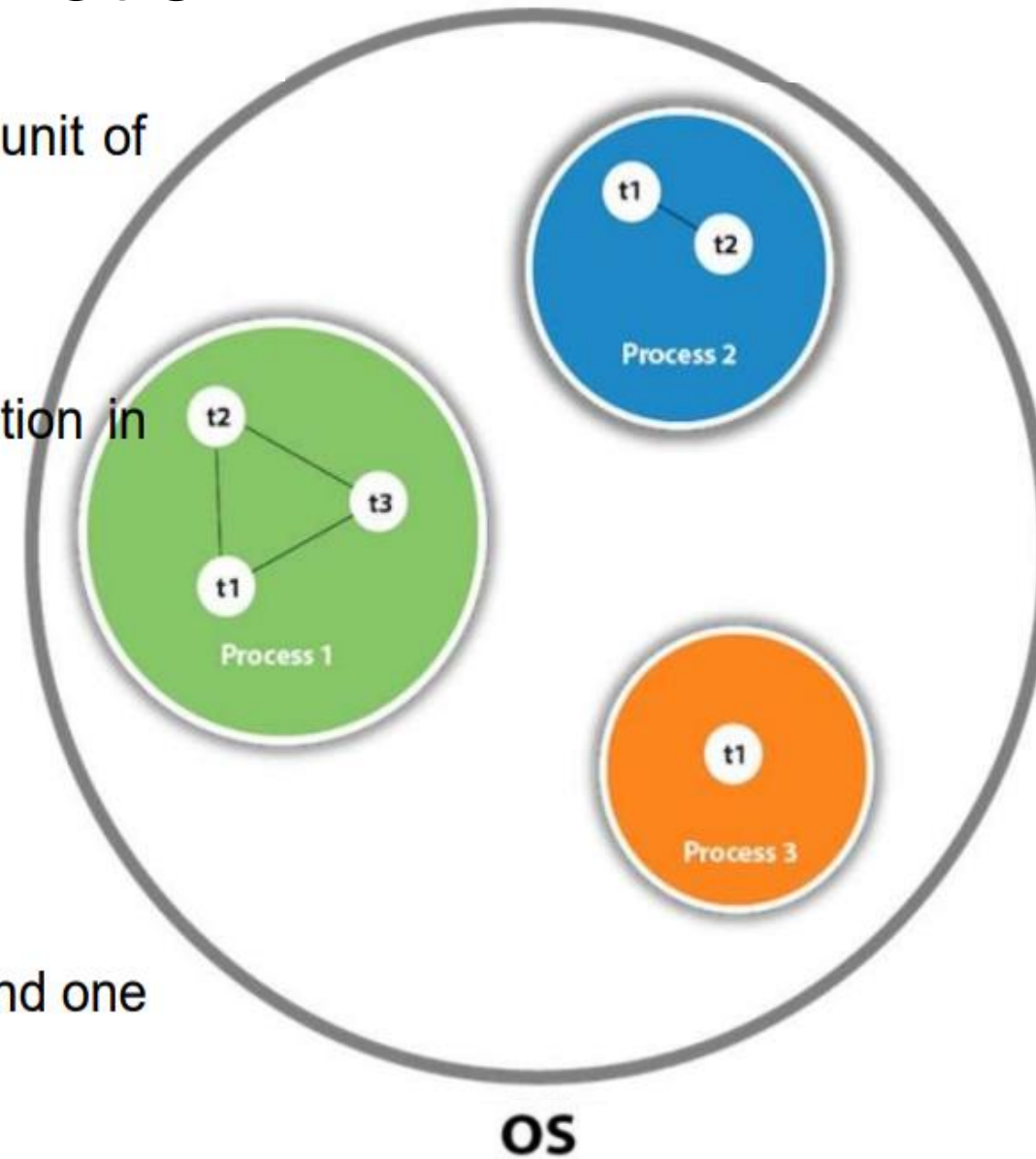
Thread-based Multitasking (Multithreading)

- › Threads share the same address space.
- › A thread is lightweight.
- › Cost of communication between the thread is low.
- › Inter-thread communication is less expensive compared to inter-process communication.

Java Thread Model

A thread is a lightweight subprocess, the smallest unit of processing. It is a separate path of execution.

- Threads are independent. If there occurs exception in one thread, it doesn't affect other threads.
- It uses a shared memory area.
- A thread is executed inside the process.
- There is context-switching between the threads.
- There can be multiple processes inside the OS, and one process can have multiple threads.

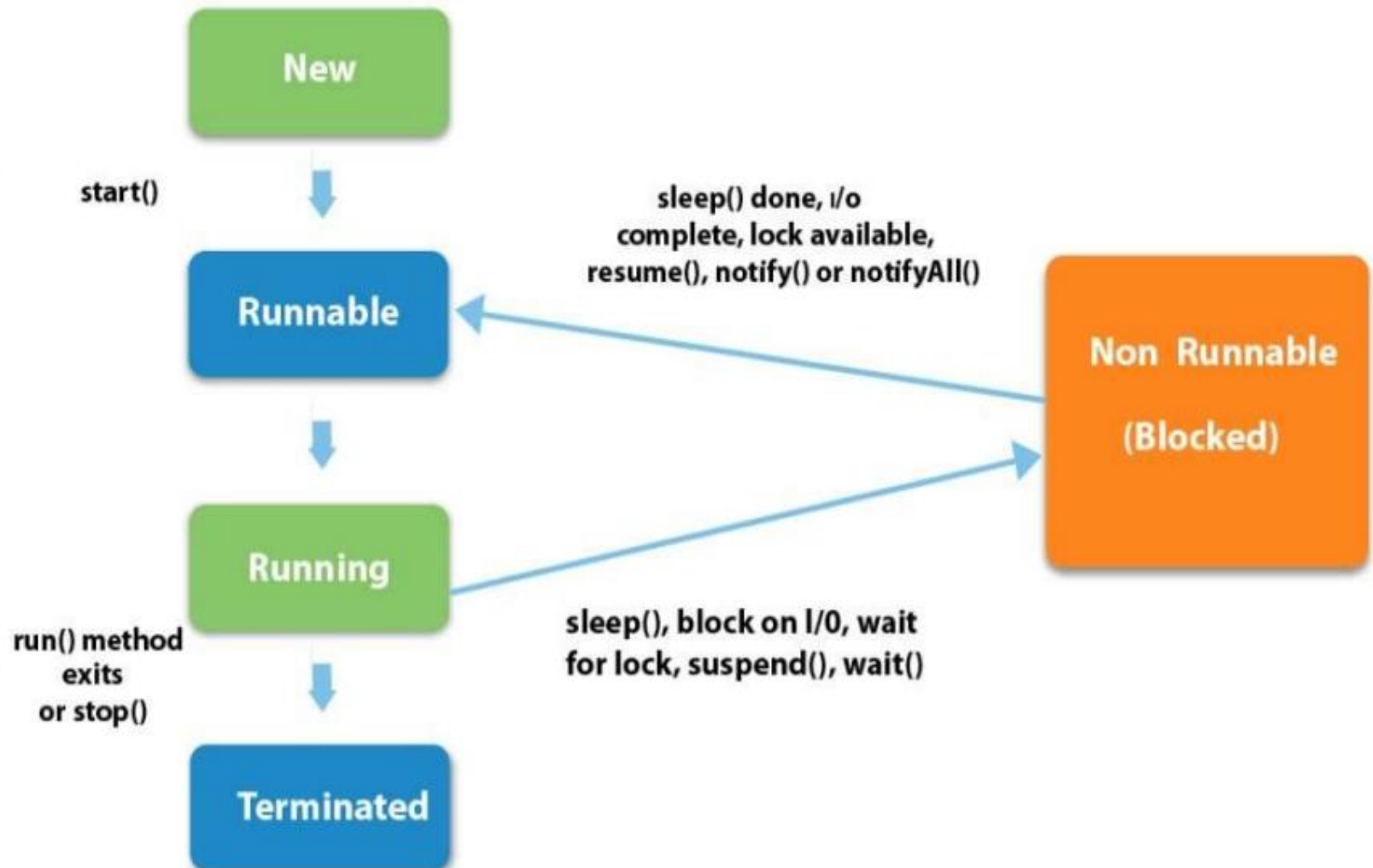


Life cycle of a Thread (Thread States)

The life cycle of the thread in java is controlled by JVM.

The java thread states are as follows:

1. New
2. Runnable
3. Running
4. Non-Runnable (Blocked)
5. Terminated



Life cycle of a Thread (Thread States)

1) New

The thread is in new state if you create an instance of Thread class but before the invocation of start() method.

2) Runnable

The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.

3) Running

The thread is in running state if the thread scheduler has selected it.

4) Non-Runnable (Blocked)

This is the state when the thread is still alive, but is currently not eligible to run.

5) Terminated

A thread is in terminated or dead state when its run() method exits.

Main thread in Java

When a Java program starts up, one thread begins running immediately. This is usually called the **main thread** of program, because it is the one that is executed when program begins.

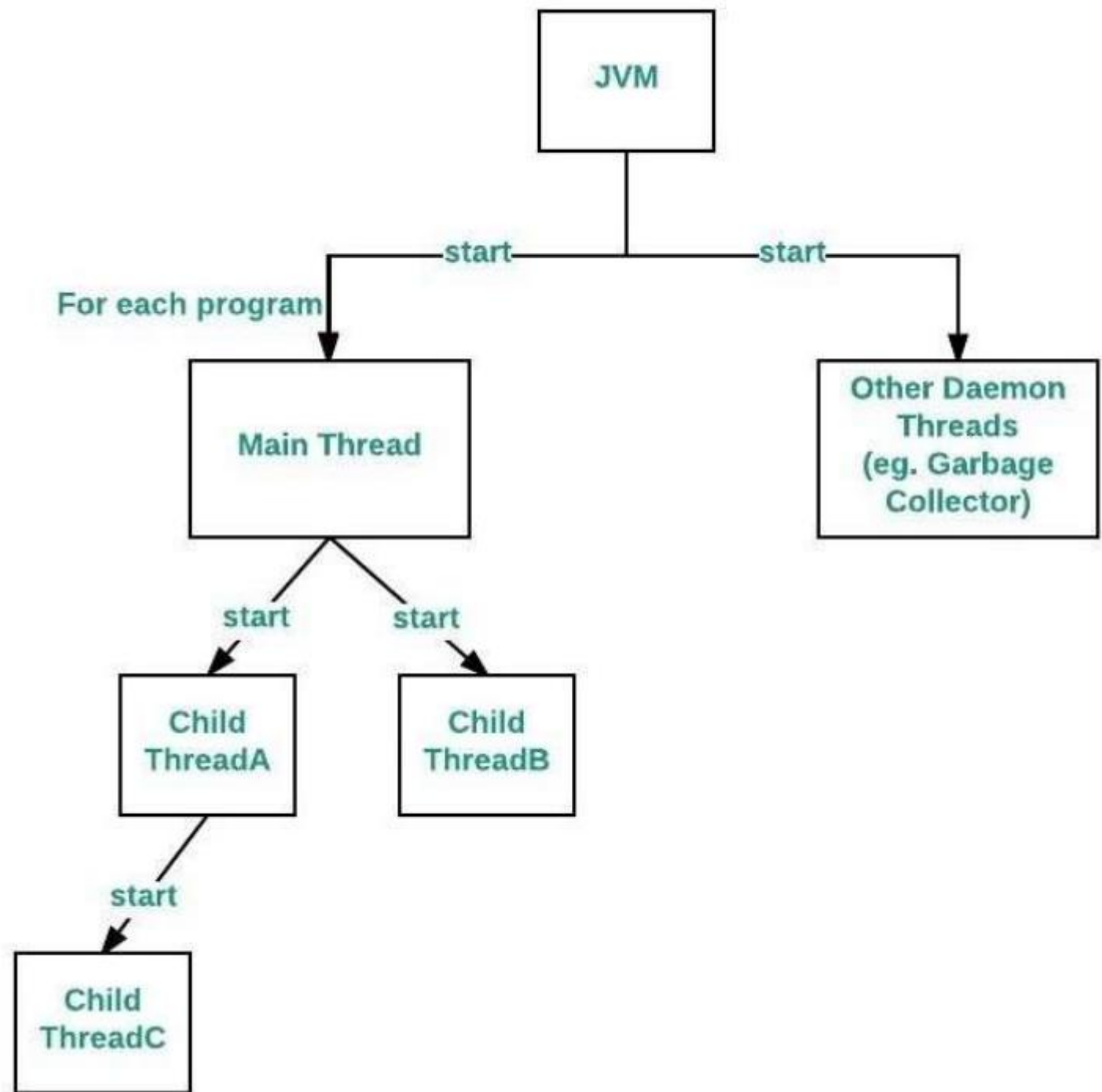
Properties :

- It is the thread from which other “**child**” **threads** will be spawned.
- Often, it must be the last thread to finish execution because it performs various shutdown actions

How to control Main thread

- To control main thread, we must obtain a **reference** to it.
- This can be done by calling the method ***currentThread()*** which is present in **Thread class**.
- This method returns a reference to the thread on which it is called.

Main thread in Java



Main thread Example

```
class currentthread
{
    public static void main(String args[])
    {
        Thread t=Thread.currentThread();
        System.out.println("Current Thread:"+t);

        t.setName("My Thread");
        System.out.println("After changing name:"+t.getName());
        System.out.println("Its priority is:"+t.getPriority());

        try
        {
            for(int i=0;i<5;i++)
            {
                System.out.println(i);
                Thread.sleep(100);
            }
        }
        catch (InterruptedException e)
        {System.out.println("Main thread interrupted");}
    }
}
```

Output:

```
Current Thread:Thread[main,5,main]
After changing name:My Thread
Its priority is:5
0
1
2
3
4
```

Java Thread class

- Java provides Thread class to achieve thread programming.
- Thread class provides constructors and methods to create and perform operations on a thread.
- Thread class extends Object class and implements Runnable interface.

Java Thread Methods

S.N.	Modifier and Type	Method	Description
1)	void	<code>start()</code>	It is used to start the execution of the thread.
2)	void	<code>run()</code>	It is used to do an action for a thread.
3)	static void	<code>sleep()</code>	It sleeps a thread for the specified amount of time.

Java Thread class Methods

4)	static Thread	<code>currentThread()</code>	It returns a reference to the currently executing thread object.
5)	void	<code>join()</code>	It waits for a thread to die.
6)	int	<code>getPriority()</code>	It returns the priority of the thread.
7)	void	<code>setPriority()</code>	It changes the priority of the thread.
8)	String	<code>getName()</code>	It returns the name of the thread.
9)	void	<code>setName()</code>	It changes the name of the thread.
10)	long	<code>getId()</code>	It returns the id of the thread.
11)	boolean	<code>isAlive()</code>	It tests if the thread is alive.

Java Thread class Methods

12)	static void	<code>yield()</code>	It causes the currently executing thread object to pause and allow other threads to execute temporarily.
13)	void	<code>suspend()</code>	It is used to suspend the thread.
14)	void	<code>resume()</code>	It is used to resume the suspended thread.
15)	void	<code>stop()</code>	It is used to stop the thread.
16)	void	<code>destroy()</code>	It is used to destroy the thread group and all of its subgroups.
17)	void	<code>notify()</code>	It is used to give the notification for only one thread which is waiting for a particular object.
18)	void	<code>notifyAll()</code>	It is used to give the notification to all waiting threads of a particular object.

How to create thread

There are two ways to create a thread:

- By implementing Runnable interface.
- By extending Thread class.

Thread class:

Thread class provide constructors and methods to create and perform operations on a thread.

Thread class **extends Object** class and **implements Runnable** interface.

Commonly used Constructors of Thread class:

Thread()

Thread(String name)

Thread(Runnable r)

Thread(Runnable r,String name)

Implementing the Runnable Interface

The easiest way to create a thread is to create a class that implements the runnable interface. After implementing runnable interface, the class needs to implement the `run()` method.

Run Method Syntax: **`public void run()`**

It introduces a concurrent thread into your program. This thread will end when `run()` method terminates.

- You must specify the code that your thread will execute inside `run()` method.
- `run()` method can call other methods, can use other classes and declare variables just like other normal method.

Implementing the Runnable Interface

The **Runnable interface** should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named `run()` and is used to perform action for a thread.

Starting a thread:

start() method of Thread class is used to start a newly created thread. It performs following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target `run()` method will run.

Implementing the Runnable Interface

Steps :

1. Declare the class as implementing Runnable interface.
2. Implement the `run()` method.
3. Create a thread by defining an object that is instantiated from the runnable interface implemented class as the target of the thread.
4. Call the thread's `start()` method to run the thread.

Example:

```
Mythread m=new Mythread();
```

```
Thread T=new Thread(m);
```

```
T.start();
```

Or

```
new Thread(new Mythread()).start();
```

Implementing the Runnable Interface

Note:

If you are implementing Runnable interface in your class, then you need to explicitly create a Thread class object and need to pass the Runnable interface implemented class object as a parameter in its constructor.

```
class MyThread implements Runnable
{
    public void run()
    {
        System.out.println("concurrent thread started running..");
    }
}

class MyThreadDemo
{
    public static void main(String args[])
    {
        MyThread mt = new MyThread();
        Thread t = new Thread(mt);
        t.start();
    }
}
```

Output:

```
concurrent thread started running..
```

Extending Thread class

This is another way to create a thread by a new class that extends Thread class and create an instance of that class.

The extending class must override run() method which is the entry point of new thread.

Steps :

1. Declare the class as extending the Thread class.
2. Implement the **run()** method.
3. Create a thread object.
4. Call the thread's **start()** method to initiate the thread execution.

Extending Thread class

In this case also, we must override the run() and then use the start() method to run the thread.

Also, when you create MyThread class object, Thread class constructor will also be invoked, as it is the super class, hence MyThread class object acts as Thread class object.

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("concurrent thread started running..");
    }
}

class MyThreadDemo
{
    public static void main(String args[])
    {
        MyThread mt = new MyThread();
        mt.start();
    }
}
```

Output:

```
concurrent thread started running..
```


Extending Thread class

Some Important points to Remember

- When we extend Thread class, we cannot override ***setName()*** and ***getName()*** functions, because they are declared final in Thread class.
- While using sleep(), always handle the exception it throws.

static void sleep(long milliseconds) throws InterruptedException

Questions :

1. What if we call run() method directly without using start() method?
2. Can we Start a thread twice?

isAlive() and join() methods of Thread Class

Q) How can one thread know when another thread has ended?

A) Java multi-threading provides two ways to find that

1) **isAlive()**

Syntax :

final boolean isAlive()

Return Value:

Returns true if the thread upon which it is called is still running.

It returns false otherwise.

2) **join()**

Syntax :

final void join() throws InterruptedException

isAlive() and join() methods of Thread Class

isAlive() :

- It tests if this thread is alive.
- A thread is alive if it has been started and has not yet died.
- There is a transitional period from when a thread is running to when a thread is not running. After the run() method returns, there is a short period of time before the thread stops.
- If we want to know if the start method of the thread has been called or if thread has been terminated, we must use isAlive() method.
- This method is used to find out if a thread has actually been started and has yet not terminated.

isAlive() and join() methods of Thread Class

join() :

- When the join() method is called, the current thread will simply wait until the thread it is joining with is no longer alive.
- Or we can say the method that you will more commonly use to wait for a thread to finish is called join().
- This method waits until the thread on which it is called terminates.
- Its name comes from the concept of the calling thread waiting until the specified thread joins it.
- Additional forms of join() allow you to specify a maximum amount of time that you want to wait for the specified thread to terminate.

Syntax: *final void join(long milliseconds) throws InterruptedException*

isAlive() and join() methods of Thread Class

```
class joinandalive1 extends Thread
{
    public void run()
    {
        System.out.println("Hi thread1..");

        try
        {
            Thread.sleep(500);
        } catch (InterruptedException e) {}
        System.out.println("Exiting thread1...");
    }
}

class joinandalive2 extends Thread
{
    public void run()
    {
        System.out.println("Hi thread2..");

        try
        {
            Thread.sleep(1000);
        } catch (InterruptedException e) {}
        System.out.println("Exiting thread2...");
    }
}
```

```
class joinandalive
{
    public static void main(String args[])
    {
        joinandalive1 j1=new joinandalive1();
        joinandalive2 j2=new joinandalive2();
        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        j1.start();
        j2.start();

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());

        /*
        //This is the first method to make main thread to finish its execution last
        try{Thread.sleep(1000);}catch(InterruptedException e){}*/

        // Here main may exit before thread2 but waits for thread1 to finish
        try
        {
            j1.join();
        } catch (InterruptedException e) {}

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        System.out.println("Main thread exiting..");
    }
}
```

isAlive() and join() methods of Thread Class

Output:

```
false
false
true
true
Hi thread1..
Hi thread2..
Exiting thread1...
false
true
Main thread exiting..
Exiting thread2...
```

```
class joinandalive
{
    public static void main(String args[])
    {
        joinandalive1 j1=new joinandalive1();
        joinandalive2 j2=new joinandalive2();
        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        j1.start();
        j2.start();

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());

        //Here main thread waits for both thread1 and thread2 to finish
        try
        {
            j1.join();
            j2.join();
        }catch (InterruptedException e){}

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        System.out.println("Main thread exiting..");
    }
}
```

Output:

```
false
false
true
true
Hi thread1..
Hi thread2..
Exiting thread1...
Exiting thread2...
false
false
Main thread exiting..
```


Synchronization

Synchronization in java is the capability to control the access of multiple threads to any shared resource.

Java Synchronization is better option where we want to allow only one thread to access the shared resource.

The synchronization is mainly used to

- To prevent thread interference.
- To prevent consistency problem.

Synchronization

Why we need Synchronization?

If we do not use synchronization, and let two or more threads access a shared resource at the same time, it will lead to distorted results.

Consider an example, Suppose we have two different threads T1 and T2, T1 starts execution and **save certain values in a file temporary.txt** which will be used to calculate some result when T1 returns. Meanwhile, T2 starts and before T1 returns, **T2 change the values saved by T1 in the file temporary.txt** (temporary.txt is the shared resource). Now obviously T1 will return wrong result.

To prevent such problems, synchronization was introduced. With synchronization in above case, once T1 starts using temporary.txt file, this file will be locked(LOCK mode), and no other thread will be able to access or modify it until T1 returns.

Types of Thread Synchronization

There are two types of thread synchronization.

1. Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data.

a. Synchronized method.

b. Synchronized block.

2. Cooperation (Inter-thread communication)

It is all about allowing synchronized threads to communicate with each other.

Understanding the problem without Synchronization

```
class First
{
    public void display(String msg)
    {
        System.out.print ("["+msg);
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
        System.out.println ("]");
    }
}
```

```
class Second extends Thread
{
    String msg;
    First fobj;
    Second (First fp,String str)
    {
        fobj = fp;
        msg = str;
        start();
    }
    public void run()
    {
        fobj.display(msg);
    }
}
```

```
public class Main
{
    public static void main (String[] args)
    {
        First fnew = new First();
        Second ss = new Second(fnew, "welcome");
        Second ss1= new Second(fnew,"new");
        Second ss2 = new Second(fnew, "programmer");
    }
}
```

Output:

```
[ new[welcome[programmer]
]
```

Java synchronized method

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

➤ **Syntax :**

synchronized method_header

{

// Block of code

}

Understanding the problem with Synchronized method

```
class First
{
    synchronized public void display(String msg)
    {
        System.out.print ("["+msg);
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
        System.out.println ("]");
    }
}
```

```
class Second extends Thread
{
    String msg;
    First fobj;
    Second (First fp,String str)
    {
        fobj = fp;
        msg = str;
        start();
    }
    public void run()
    {
        fobj.display(msg);
    }
}
```

```
public class Main
{
    public static void main (String[] args)
    {
        First fnew = new First();
        Second ss = new Second(fnew, "welcome");
        Second ss1= new Second(fnew,"new");
        Second ss2 = new Second(fnew, "programmer");
    }
}
```

Output:

```
[welcome]
[programmer]
[new]
```


Using Synchronized block

- If want to synchronize access to an object of a class or only a part of a method to be synchronized then we can use synchronized block for it.
- It is capable to make any part of the object and method synchronized.
- Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.
- If you put all the codes of the method in the synchronized block, it will work same as the synchronized method.

➤ **Syntax:**

```
synchronized (object)  
{  
    // statements to be synchronized  
}
```

Using Synchronized block

```
class First
{
    public void display(String msg)
    {
        System.out.print ("["+msg);
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
        System.out.println ("]");
    }
}

class Second extends Thread
{
    String msg;
    First fobj;
    Second (First fp,String str)
    {
        fobj = fp;
        msg = str;
        start();
    }
    public void run()
    {
        synchronized(fobj)    //Synchronized block
        {
            fobj.display(msg);
        }
    }
}
```

```
public class Main
{
    public static void main (String[] args)
    {
        First fnew = new First();
        Second ss = new Second(fnew, "welcome");
        Second ss1= new Second (fnew,"new");
        Second ss2 = new Second(fnew, "programmer");
    }
}
```

Output:

```
[welcome]
[programmer]
[new]
```

Difference between synchronized keyword and synchronized block

When we use synchronized keyword with a method, it acquires a lock in the object for the whole method. It means that no other thread can use any synchronized method until the current thread, which has invoked its synchronized method, has finished its execution.

synchronized block acquires a lock in the object only between parentheses after the synchronized keyword. This means that no other thread can acquire a lock on the locked object until the synchronized block exits. But other threads can access the rest of the code of the method.

Q) Which is more preferred - Synchronized method or Synchronized block?

Inter-thread Communication

- Inter-thread communication or **Co-operation** is all about allowing synchronized threads to communicate with each other.
- Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its **critical section** and another thread is allowed to enter (or lock) in the same critical section to be executed.
- A critical section is a block of code that accesses a **shared resource** and **can't be executed by more than one thread** at the same time.

Inter-thread Communication

➤ It is implemented by following methods of Object class:

wait() : tells calling thread to give up monitor and go to sleep until some other thread enters the same monitor and call notify.

notify(): wakes up a thread that called wait() on same object.

notifyAll(): wakes up all the thread that called wait() on same object.

All the three method can be called only from within a synchronized context

Inter-thread Communication

Syntaxes :

wait():

public final void wait()throws InterruptedException

public final void wait(long timeout) throws InterruptedException

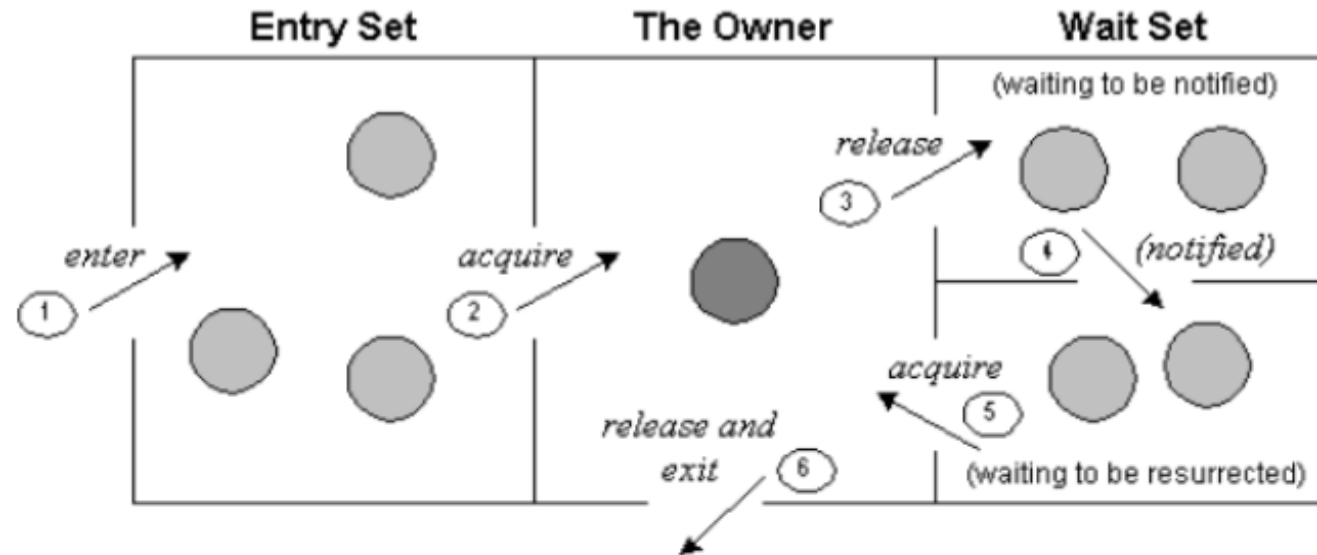
notify():

public final void notify()

notifyAll():

public final void notifyAll()

Understanding the process of inter-thread communication



The point to point explanation of the above diagram is as follows:

1. Threads enter to acquire lock.
2. Lock is acquired by one thread.
3. Now thread goes to waiting state if you call wait() method on the object. Otherwise it releases the lock and exits.
4. If you call notify() or notifyAll() method, thread moves to the notified state (runnable state).
5. Now thread is available to acquire lock.
6. After completion of the task, thread releases the lock and exits the monitor state of the object.

Inter-thread Communication

```
class Customer
{
    int amount=10000;

    synchronized void withdraw(int amount){
        System.out.println("going to withdraw...");

        if(this.amount<amount){
            System.out.println("Less balance; waiting for deposit...");
            try
            {wait();}
            catch(Exception e){}
        }
        this.amount-=amount;
        System.out.println("withdraw completed...");
    }

    synchronized void deposit(int amount){
        System.out.println("going to deposit...");
        this.amount+=amount;
        System.out.println("deposit completed... ");
        notify();
    }
}
```

```
class Test{
    public static void main(String args[]){
        final Customer c=new Customer();
        new Thread(){
            public void run(){c.withdraw(15000);}
        }.start();
        new Thread(){
            public void run(){c.deposit(10000);}
        }.start();
    }
}
```

Output:

```
going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed...
```

Differences between wait() and sleep()

wait()	sleep()
wait() method releases the lock	sleep() method doesn't release the lock.
is the method of Object class	is the method of Thread class
is the non-static method	is the static method
is the non-static method	is the static method
should be notified by notify() or notifyAll() methods	after the specified amount of time, sleep is completed.
called from synchronised block	no such requirement
wait() is generally used on condition	sleep() method is simply used to put your thread on sleep.



Unit 3

1. The Collection Framework

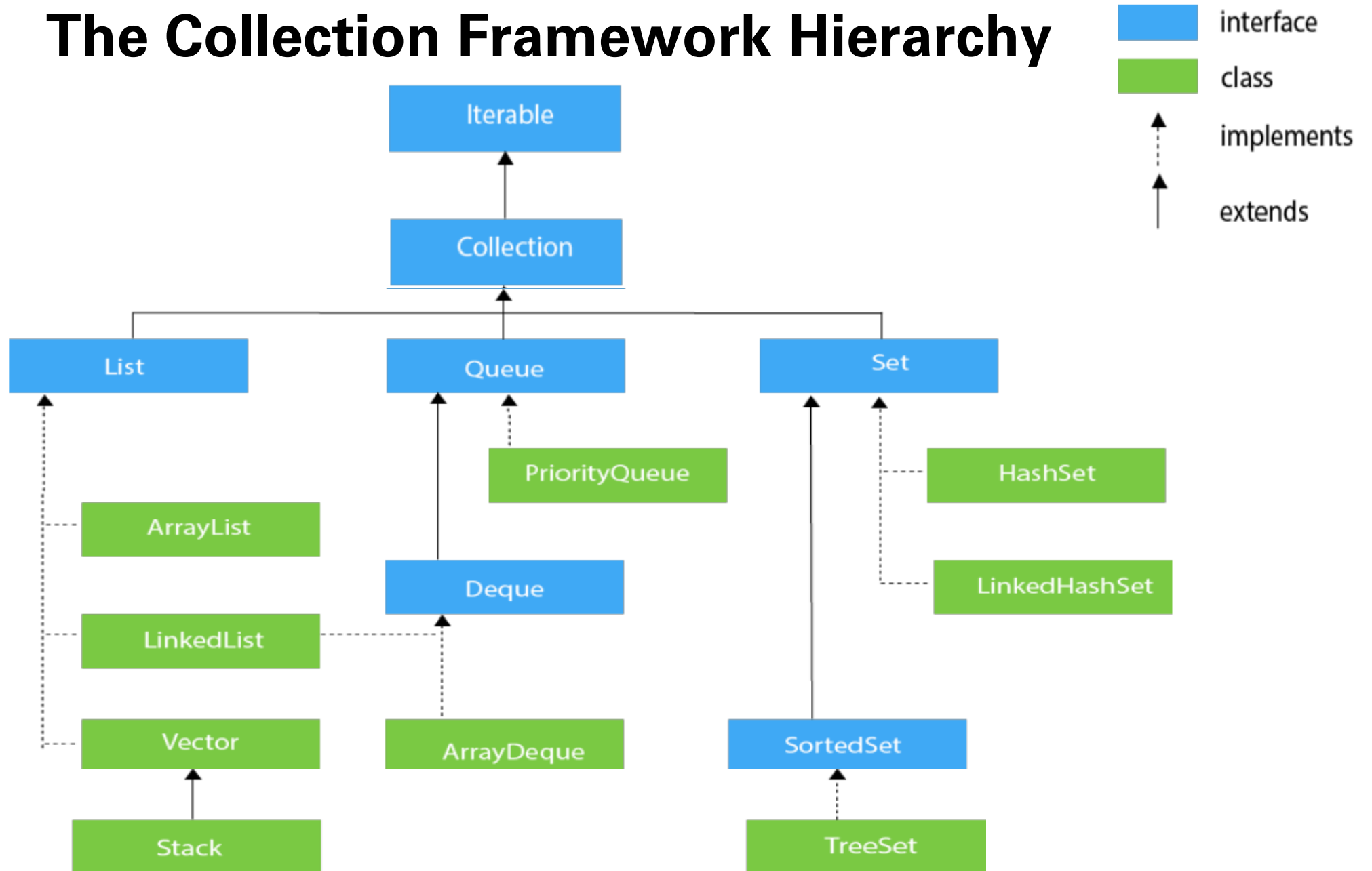
The Collection Framework

- The Collection Framework in Java is a framework that provides an architecture to store and manipulate the group of objects.
- Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- Java Collection framework provides many **interfaces** (Set, List, Queue, Deque) and **classes** (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).
- The **java.util** package contains all the classes and interfaces for the Collection framework.

Collection Interface

- The Collection interface is the interface which is implemented by all the classes in the collection framework.
- In other words, we can say that the **Collection interface builds the foundation on which the collection framework depends.**

The Collection Framework Hierarchy



The List Interface

- List interface is the **child interface of Collection interface**.
- It inhibits a list type data structure in which we can store the ordered collection of objects. It can have duplicate values.
- List interface is implemented by the classes **ArrayList**, **LinkedList**, **Vector**, and **Stack**.
- To instantiate the List interface, we must use :
List <data-type> list1= new ArrayList();
List <data-type> list2 = new LinkedList();
- There are various methods in List interface that can be used to insert, delete, and access the elements from the list.



The List Interface Methods

Methods	Description
<code>add()</code>	adds an element to a list
<code>addAll()</code>	adds all elements of one list to another
<code>get()</code>	helps to randomly access elements from lists
<code>iterator()</code>	returns <code>iterator</code> object that can be used to sequentially access elements of lists
<code>set()</code>	changes elements of lists
<code>remove()</code>	removes an element from the list
<code>removeAll()</code>	removes all the elements from the list
<code>clear()</code>	removes all the elements from the list (more efficient than <code>removeAll()</code>)
<code>size()</code>	returns the length of lists
<code>toArray()</code>	converts a list into an array
<code>contains()</code>	returns <code>true</code> if a list contains specific element

The ArrayList Class

- The ArrayList class implements the List interface.
- It uses a dynamic array to store the duplicate element of different data types.
- The ArrayList class maintains the insertion order and is non-synchronized.
- The elements stored in the ArrayList class can be randomly accessed.

Operations in a Java ArrayList Interface

- **Operation 1:** Adding elements to List class using add() method
- **Operation 2:** Updating elements in List class using set() method
- **Operation 3:** Searching for elements using indexOf(), lastIndexOf methods
- **Operation 4:** Removing elements using remove() method
- **Operation 5:** Accessing Elements in List class using get() method
- **Operation 6:** Checking if an element is present in the List class using contains() method

The ArrayList Class

```
import java.util.ArrayList;
import java.util.Iterator;

public class ArrayListDemo {
    public static void main(String[] args) {
        // Creating an ArrayList
        ArrayList<String> list = new ArrayList<>();

        // Adding elements using add()
        list.add("Java");
        list.add("Python");
        list.add("C++");
        list.add("JavaScript");
        list.add(3, "Enterprise Java");
        list.add("Java");
        System.out.println("The index of C++ is " + list.indexOf("C++"));
        System.out.println("The last index of Java is " + list.lastIndexOf("Java")
            );

        // Adding a collection using addAll()
        ArrayList<String> anotherList = new ArrayList<>();
        anotherList.add("Ruby");
        anotherList.add("Swift");
        list.addAll(anotherList);
    }
}
```

The ArrayList Class

```
// Accessing elements using get()
System.out.println("Element at index 2: " + list.get(2));
// Iterating over elements using iterator()
System.out.println("Iterating over elements:");
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
// Modifying an element using set()
list.set(3, "TypeScript");
System.out.println("Does the list contain 'C++'? " + list.contains("C++"));
list.remove("C++"); // Removing an element using remove()

list.removeAll(anotherList); // Removing a collection using removeAll()

list.clear(); // Clearing the ArrayList using clear()

// Checking the size of the ArrayList using size()
System.out.println("Size of the ArrayList: " + list.size());

// Converting ArrayList to Array using toArray()
String[] array = list.toArray(new String[0]);

// Checking if an element exists using contains()
System.out.println("Does the list contain 'Java'? " + list.contains("Java"));
}); }
```

The ArrayList Class

Output

```
The index of C++ is 2
The last index of Java is 5
Element at index 2: C++
Iterating over elements:
Java
Python
C++
Enterprise Java
JavaScript
Java
Ruby
Swift
Does the list contain 'C++'? true
Size of the ArrayList: 0
Does the list contain 'Java'? false
```


The LinkedList Class

- LinkedList implements the Collection interface.
- It uses a doubly linked list internally to store the elements.
- It can store the duplicate elements.
- It maintains the insertion order and is not synchronized.

```
import java.util.LinkedList;
import java.util.Iterator;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> linkedList = new LinkedList<>();

        // Adding elements using add()
        linkedList.add("Java");
        linkedList.add("Python");
        linkedList.add("C++");
        linkedList.add("JavaScript");
    }
}
```

The LinkedList Class

```
// Adding a collection using addAll()
LinkedList<String> anotherList = new LinkedList<>();
anotherList.add("Ruby");
anotherList.add("Swift");
linkedList.addAll(anotherList);

System.out.println("Element at index 2: " + linkedList.get(2));
System.out.println("Iterating over elements:");
Iterator<String> iterator = linkedList.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
linkedList.set(3, "TypeScript");
linkedList.remove("C++");
linkedList.removeAll(anotherList);
linkedList.clear();

System.out.println("Size of the LinkedList: " + linkedList.size());
String[] array = linkedList.toArray(new String[0]);
System.out.println("Does the list contain 'Java'? " + linkedList.contains
    ("Java"));
}
}
```

The LinkedList Class

Output

```
Element at index 2: C++  
Iterating over elements:  
Java  
Python  
C++  
JavaScript  
Ruby  
Swift  
Size of the LinkedList: 0  
Does the list contain 'Java'? false  
|
```


The LinkedList Class Vs ArrayList class

1. Underlying Data Structure:

- **LinkedList:** Internally implemented as a doubly-linked list where each element is stored as a separate object (node) containing a reference to the previous and next elements in the list.
- **ArrayList:** Internally implemented as a dynamic array that can dynamically resize itself when elements are added or removed.

2. Performance:

- **LinkedList:** Efficient for frequent insertions and deletions at arbitrary positions in the list, as it involves only adjusting the references of neighboring nodes. However, accessing elements by index is less efficient compared to ArrayList because it requires traversing the list from the beginning or end.
- **ArrayList:** Efficient for random access and retrieval of elements by index since it uses an array-based data structure. However, insertion and deletion operations at arbitrary positions can be less efficient, especially when resizing the underlying array is required.

3. Memory Overhead:

- **LinkedList:** Each element in the list requires additional memory overhead for storing references to the previous and next elements.
- **ArrayList:** Generally consumes less memory overhead since it stores elements in a contiguous array, without the need for additional references between elements.

The LinkedList Class Vs ArrayList class

4. Insertion and Deletion:

- **LinkedList:** Provides efficient insertion and deletion operations, especially when performed at the beginning or end of the list or when the position is known.
- **ArrayList:** Insertion and deletion operations may be less efficient, particularly when performed frequently or at arbitrary positions, as it may require shifting elements within the underlying array.

5. Iterating Over Elements:

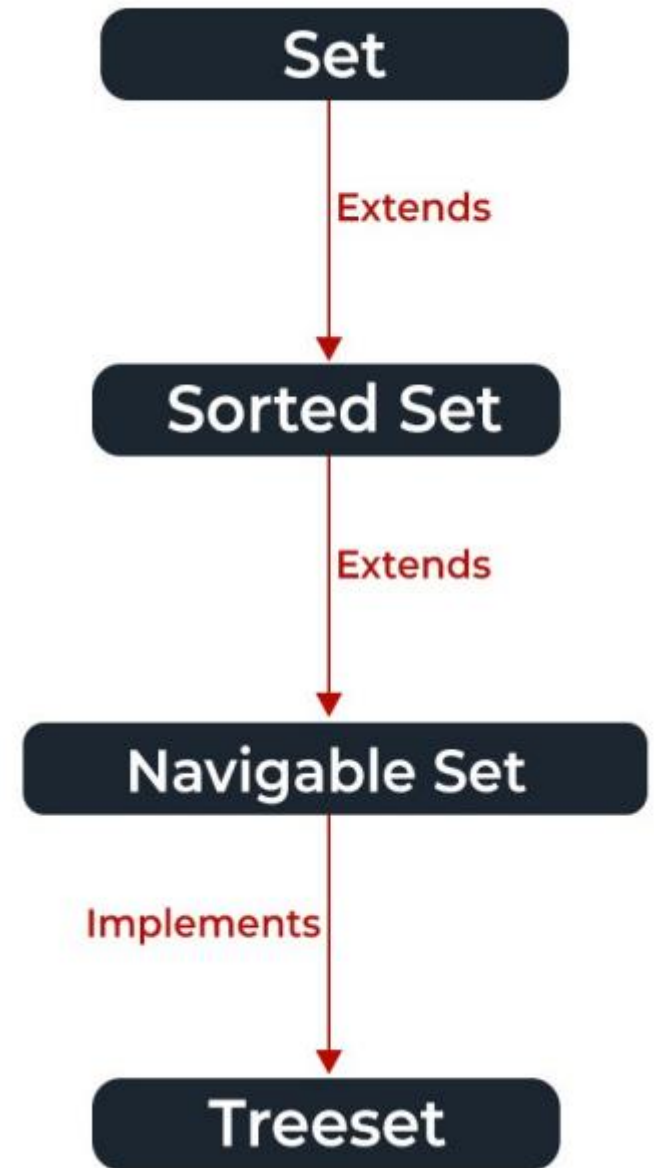
- **LinkedList:** Iterating over elements using an iterator can be efficient, especially for sequential traversal.
- **ArrayList:** Iterating over elements is generally efficient, as it provides direct access to elements using indexes.

6. Use Cases:

- **LinkedList:** Suitable for scenarios requiring frequent insertions and deletions, such as queues or certain types of linked data structures.
- **ArrayList:** Suitable for scenarios requiring fast random access and retrieval, such as maintaining a collection of elements where elements are frequently accessed by their index.

The Set Interface

- The set interface is present in `java.util` package and extends the `Collection` interface.
- It is an `unordered` collection of objects in which duplicate values cannot be stored.
- It is an interface that implements the mathematical set.
- This interface contains the methods inherited from the `Collection` interface and adds a feature that `restricts` the insertion of the duplicate elements.
- There are two interfaces that extend the set implementation namely `SortedSet` and `NavigableSet`.
- The class which implements the navigable set is a `TreeSet` which is an implementation of a self-balancing tree.
- Therefore, this interface provides us with a way to navigate through this tree.



The Set Interface

Method	Description
<code><u>add(element)</u></code>	This method is used to add a specific element to the set. The function adds the element only if the specified element is not already present in the set else the function returns False if the element is already present in the Set.
<code><u>addAll(collection)</u></code>	This method is used to append all of the elements from the mentioned collection to the existing set. The elements are added randomly without following any specific order.
<code><u>clear()</u></code>	This method is used to remove all the elements from the set but not delete the set. The reference for the set still exists.
<code><u>contains(element)</u></code>	This method is used to check whether a specific element is present in the Set or not.
<code><u>containsAll(collection)</u></code>	This method is used to check whether the set contains all the elements present in the given collection or not. This method returns true if the set contains all the elements and returns false if any of the elements are missing.

The Set Interface

<code>hashCode()</code>	This method is used to get the hashCode value for this instance of the Set. It returns an integer value which is the hashCode value for this instance of the Set.
<code>isEmpty()</code>	This method is used to check whether the set is empty or not.
<code>iterator()</code>	This method is used to return the iterator of the set. The elements from the set are returned in a random order.
<code>remove(element)</code>	This method is used to remove the given element from the set. This method returns True if the specified element is present in the Set otherwise it returns False.
<code>removeAll(collection)</code>	This method is used to remove all the elements from the collection which are present in the set. This method returns true if this set changed as a result of the call.
<code>retainAll(collection)</code>	This method is used to retain all the elements from the set which are mentioned in the given collection. This method returns true if this set changed as a result of the call.
<code>size()</code>	This method is used to get the size of the set. This returns an integer value which signifies the number of elements.
<code>toArray()</code>	This method is used to form an array of the same elements as that of the Set.

The Set Interface - Creating Set Objects

- Since Set is an interface, objects cannot be created of the type set. We always need a class that extends this list in order to create an object of type **HashSet** or **LinkedHashSet**.

- // Obj is the type of the object to be stored in Set and set is an object.

Set<Obj> set = new HashSet<Obj> ();

Set<Obj> set = new LinkedHashSet<Obj> ();

- **Examples:** HashSetDemo.java,
LinkedHashSetDemo.java

Operations on the Set Interface

- The set interface allows the users to perform the basic mathematical operation on the set.
 - Let's take two arrays to understand these basic operations. Let set1 = [1, 3, 2, 4, 8, 9, 0] and set2 = [1, 3, 7, 5, 4, 0, 7, 5]. Then the possible operations on the sets are:
 - **1. Intersection:** This operation returns all the common elements from the given two sets. For the above two sets, the intersection would be: **Intersection** = [0, 1, 3, 4]
 - **2. Union:** This operation adds all the elements in one set with the other. For the above two sets, the union would be: **Union** = [0, 1, 2, 3, 4, 5, 7, 8, 9]
 - **3. Difference:** This operation removes all the values present in one set from the other set. For the above two sets, the difference would be: **Difference** = [2, 8, 9]
- Example:** SetExample.java

List Vs Set Interface

List	Set
The List is an ordered sequence.	The Set is an unordered sequence.
List allows duplicate elements	Set doesn't allow duplicate elements.
Elements by their position can be accessed.	Position access to elements is not allowed.
Multiple null elements can be stored.	The null element can store only once.
List implementations are ArrayList, LinkedList, Vector, Stack	Set implementations are HashSet, LinkedHashSet.

A vertical bar on the left side of the slide with a gradient from orange at the top to blue at the bottom.

Unit 3

3. Applet Class

Java Applet

- Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantages of Applet

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

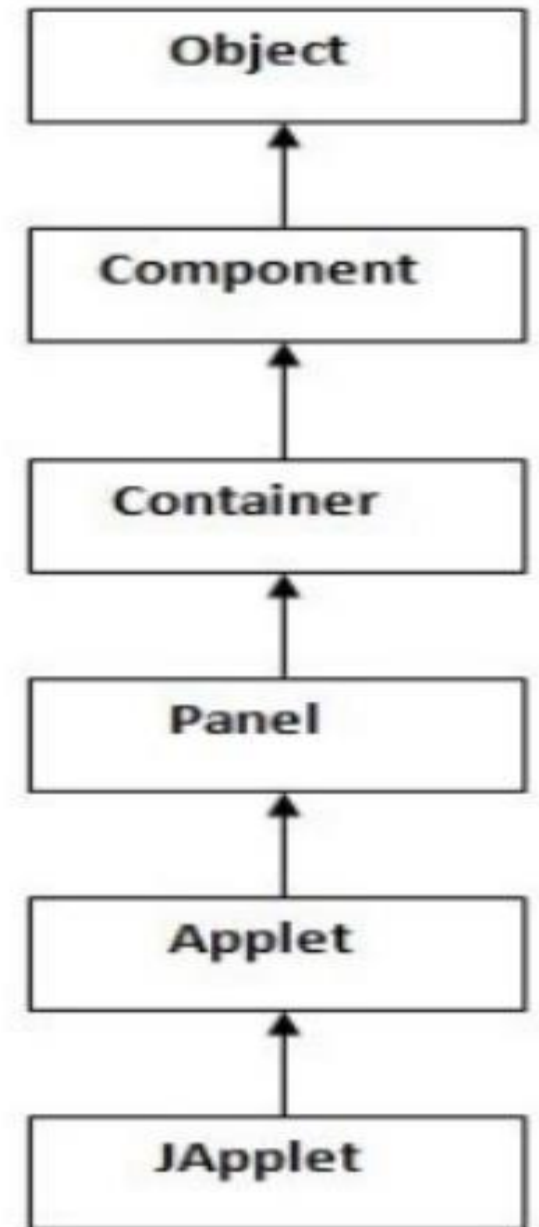
Drawback of Applet

- Plugin is required at client browser to execute applet.

Differences between an Applet and a stand-alone Java application

- A main() method is not invoked on an applet, and an applet class will not define main().
- Applets are designed to be embedded within an HTML page.
- When a user views an HTML page that contains an applet, the code for the applet is downloaded to the user's machine.
- A JVM is required to view an applet. The JVM can be either a plug-in of the Web browser or a separate runtime environment.
- The JVM on the user's machine creates an instance of the applet class and invokes various methods during the applet's lifetime.

Hierarchy of Applet



Applet Basics

- Small applications that are accessed on an Internet server, transported over the Internet, automatically installed and run as a part of a web document.
- After an applet arrives on the client, it has limited access to resources so that it can produce an arbitrary multimedia user interface.

Begins with two import statements

Import java.awt.*; :-- imports abstract window toolkit (AWT) classes

Applet interacts with user through AWT, not through console based I/O classes.

Import java.applet.*; :-- imports Applet class

All the applets that you create must be subclasses of Applet class.

Applet Life Cycle



1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.

Applet Skeleton

```
// An Applet skeleton.  
import java.awt.*;  
import java.applet.*;  
/*  
<applet code="AppletSkel" width=300 height=100:  
</applet>  
*/  
  
public class AppletSkel extends Applet {  
    // Called first.  
    public void init() {  
        // initialization  
    }  
}
```

Applet Skeleton

```
/* Called second, after init(). Also called whenever  
   the applet is restarted. */
```

```
public void start() {  
    // start or resume execution  
}
```

```
// Called when the applet is stopped.
```

```
public void stop() {  
    // suspends execution  
}
```

```
/* Called when applet is terminated. This is the last  
   method executed. */
```

```
public void destroy() {  
    // perform shutdown activities  
}
```

```
// Called when an applet's window must be restored.
```

```
public void paint(Graphics g) {  
    // redisplay contents of window  
}  
}
```

Life cycle methods for Applet

The ***java.applet.Applet*** class provides 4 life cycle methods and ***java.awt.Component*** class provides 1 life cycle method for an applet.

java.applet.Applet class

For creating any applet `java.applet.Applet` class must be inherited. It provides 4 life cycle methods of applet.

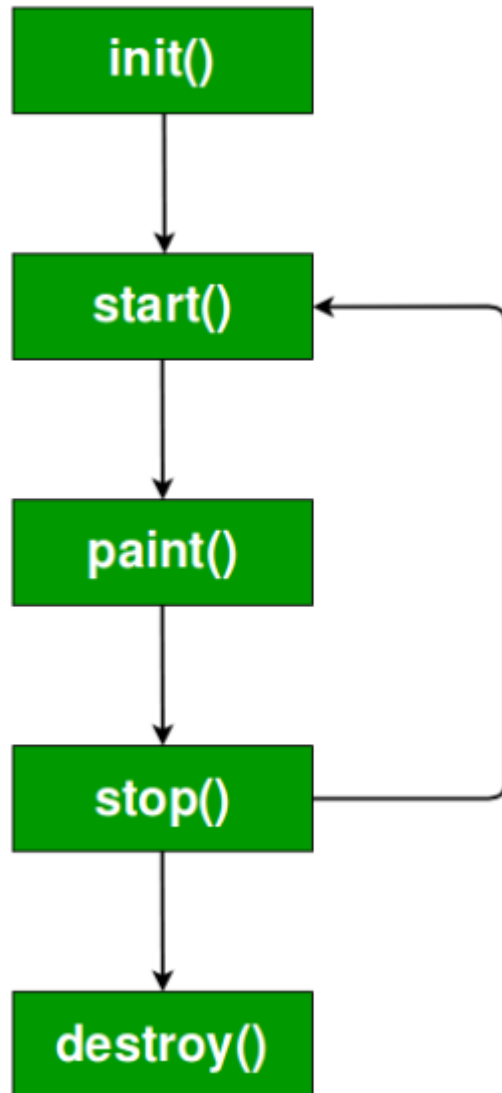
1. **public void init():** is used to initialize the Applet. It is invoked only once.
2. **public void start():** is invoked after the `init()` method or browser is maximized. It is used to start the Applet.
3. **public void stop():** is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
4. **public void destroy():** is used to destroy the Applet. It is invoked only once.

java.awt.Component class

The Component class provides 1 life cycle method of applet.

1. **public void paint(Graphics g):** is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Life cycle methods for Applet



It is important to understand the order in which the various methods shown in the above image are called.

When an applet begins, the following methods are called, in this sequence:

1. `init( )`
2. `start( )`
3. `paint( )`

When an applet is terminated, the following sequence of method calls takes place:

1. `stop( )`
2. `destroy( )`

Brief Description of Life Cycle Methods

1. **init()**: The applet's voyage starts here. In this method, the applet object is created by the browser. Because this method is called before all the other methods, programmer can utilize this method to instantiate objects, initialize variables, setting background and foreground colors in GUI etc.; the place of a constructor in an application. It is equivalent to **born state** of a thread.
2. **start()**: In `init()` method, even though applet object is created, it is in **inactive** state. An inactive applet is not eligible for microprocessor time even though the microprocessor is idle. To make the applet active, the `init()` method calls `start()` method. In `start()` method, applet becomes active and thereby eligible for processor time.

Brief Description of Life Cycle Methods

3. **paint()**: This method takes a **java.awt.Graphics** object as parameter. This class includes many methods of drawing necessary to draw on the applet window. This is the place where the programmer can write his code of what he expects from applet like animation etc. This is equivalent to **runnable state** of thread.
4. **stop()**: In this method the applet becomes temporarily inactive. An applet can come any number of times into this method in its life cycle and can go back to the active state (**paint()** method) whenever would like. It is the best place to have cleanup code. It is equivalent to the **blocked state** of the thread.
5. **destroy()**: This method is called just before an applet object is garbage collected. This is the end of the life cycle of applet. It is the best place to have cleanup code. It is equivalent to the **dead state** of the thread.

HTML <applet> Tag

We cannot open an applet directly in a browser, even though, the browser is meant to execute an applet. For every applet, a HTML file is to be written in which we give the name and address etc. of the applet.

Syntax:

```
<APPLET  
[CODEBASE = codebaseURL]  
CODE = appletFile  
[ALT = alternateText]  
[NAME = appletInstanceName]  
WIDTH = pixels HEIGHT = pixels  
[ALIGN = alignment]  
[VSPACE = pixels] [HSPACE = pixels]  
>  
[< PARAM NAME = AttributeName VALUE = AttributeValue>]  
[< PARAM NAME = AttributeName2 VALUE = AttributeValue>]  
...  
[HTML Displayed in the absence of Java]  
</APPLET>
```

HTML <applet> Tag Attributes

Attribute name	Value	Description
code	URL	It specifies the URL of Java applet class file.
width	pixels	It specifies the display width of the applet panel.
height	pixels	It specifies the display height of applet panel
align	◦ left ◦ right ◦ top ◦ middle ◦ bottom	It specifies the position of applet application relative to surrounding content.

HTML <applet> Tag Attributes

Attribute name	Value	Description
alt	text	It is used to display alternative text in case browser does not support Java.
archive	URL	This specifies the archived or compressed version of an applet application.
object	name	It specifies the URL or reference to a serialized representation of an applet.
codebase	URL	It specifies the exact or relative URL of applets .class file specified in the code attribute.
hspace	pixels	It specifies the horizontal space around the applet.
vspace	pixels	It specifies the vertical space around the applet.
name	name	It specifies the name for the applet

Supporting Browsers

Element	 Chrome	 IE	 Firefox	 Opera	 Safari
<applet>	Not supported	Not Supported	Yes	Not Supported	Yes

Creating Hello World applet :

```
// A Hello World Applet
// Save file as HelloWorld.java

import java.applet.Applet;
import java.awt.Graphics;

// HelloWorld class extends Applet
public class HelloWorld extends Applet
{
    // Overriding paint() method
    @Override
    public void paint(Graphics g)
    {
        g.drawString("Hello World", 20, 20);
    }
}
```

Notice that the applet does not have a **main()** method. Unlike Java programs, applets do not begin execution at **main()**. In fact, most applets don't even have a **main()** method. Instead, an applet begins execution when the name of its class is passed to an applet viewer or to a network browser.

Running the HelloWorld Applet :

After you enter the source code for HelloWorld.java, compile in the same way that you have been compiling java programs(using *javac* command). However, running HelloWorld with the *java* command will generate an error because it is not an application.

```
java HelloWorld
```

```
Error: Main method not found in class HelloWorld, please define the main method as:  
public static void main(String[] args)
```

There are **two** standard ways in which you can run an applet :

1. Executing the applet within a Java-compatible web browser.
2. Using an applet viewer, such as the standard tool, applet-viewer. An applet viewer executes your applet in a window. This is generally the fastest and easiest way to test your applet.

Running the HelloWorld Applet :

1. **Using java enabled web browser :** To execute an applet in a web browser we have to write a short HTML text file that contains a tag that loads the applet. We can use APPLET or OBJECT tag for this purpose. Using APPLET, here is the HTML file that executes HelloWorld :

RunHelloWorld.html

```
<html>
<body>
<applet code="HelloWorld" width="300" height="300">
</applet>
</body>
</html>
```

Running the HelloWorld Applet :

2. **Using appletviewer** : This is the easiest way to run an applet. To execute HelloWorld with an applet viewer, you may also execute the HTML file shown earlier. For example, if the preceding HTML file is saved with

RunHelloWorld.html, then the following command line will run HelloWorld :

```
appletviewer RunHelloWorld.html
```



Running the HelloWorld Applet :

3. **appletviewer with java source file** : If you include a comment at the head of your Java source code file that contains the APPLET tag then your code is documented with a prototype of the necessary HTML statements, and you can run your compiled applet merely by starting the applet viewer with your Java source code file. If you use this method, the HelloWorld source file looks like this :

```
import java.applet.Applet;  
import java.awt.Graphics;
```

```
/*  
<applet code="HelloWorld" width=200 height=60>  
</applet>  
*/
```

```
// HelloWorld class extends Applet  
public class HelloWorld extends Applet  
{  
    // Overriding paint() method  
    @Override  
    public void paint(Graphics g)  
    {  
        g.drawString("Hello World", 20, 20);  
    }  
}
```

With this approach, first compile HelloWorld.java file and then simply run below command to run applet :

appletviewer HelloWorld



PARAM Attribute

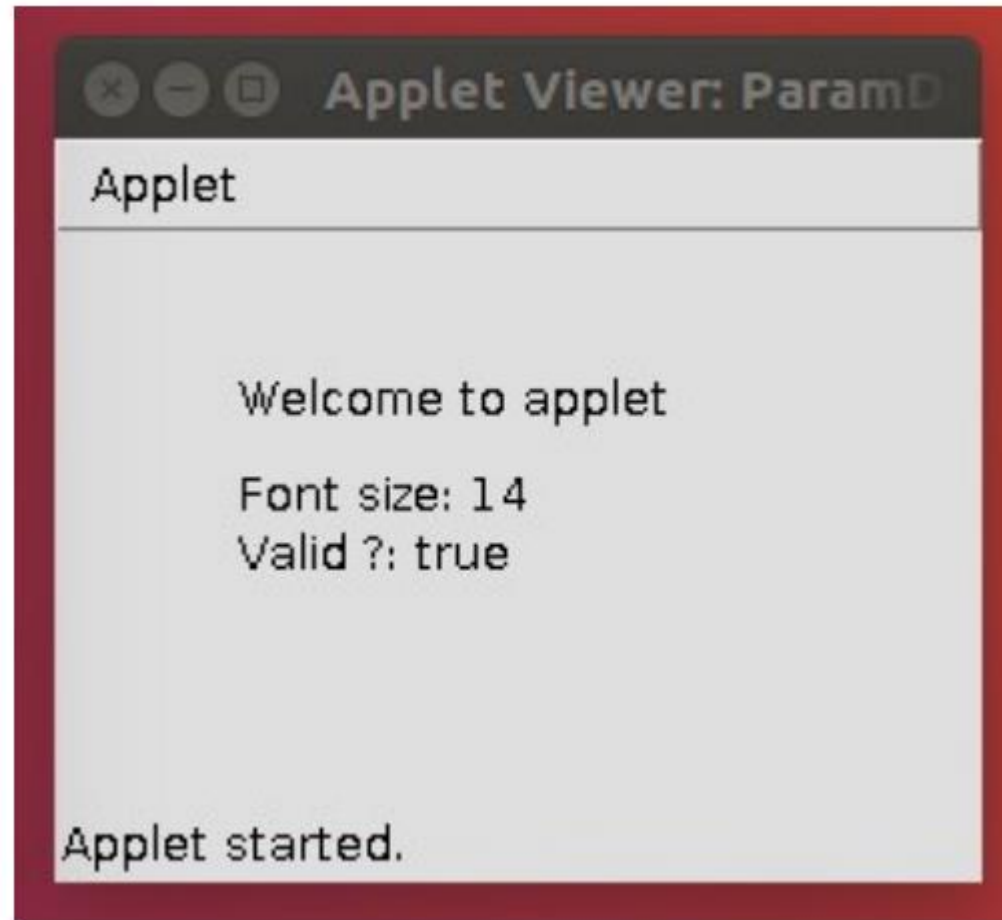
The PARAM tag allows you to specify applet-specific arguments in an HTML page. Applets access their attributes with the `getParameter( )` method.

It returns the value of the specified parameter in the form of a String object.

Thus, for numeric and boolean values, you will need to convert their string representations into their internal formats.

Example: [ParamDemo.java](#)

PARAM Attribute



getDocumentBase() and getCodeBase()

Java will allow the applet to load the data from

- > the directory holding HTML file that started applet (the document base)
- > the directory from which the applet's class file was loaded (the code base)

These directories are returned as URL(Uniform Resource Locator) objects by

- * `getDocumentBase()`
- * `getCodeBase()`

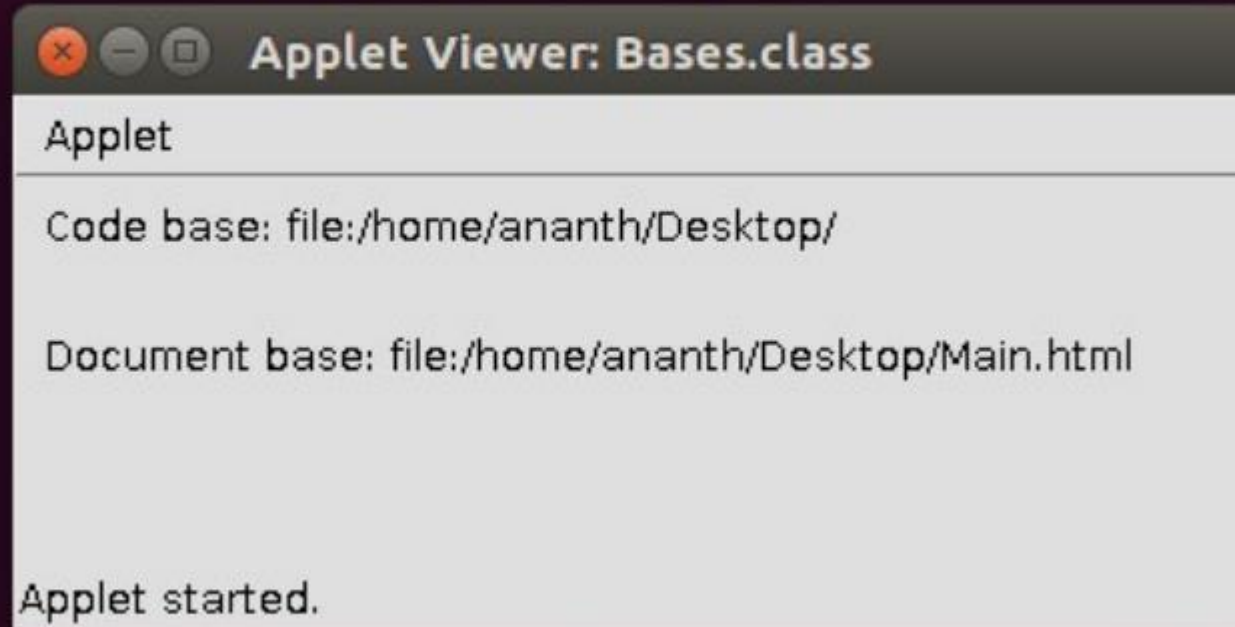
URL is a class identifies the address information on the Internet

It is defined in the package `java.net`.

Example: `Bases.java`

getDocumentBase() and getCodeBase()

```
ananth@ananth-Veriton-Series:~/Desktop$ javac Bases.java  
ananth@ananth-Veriton-Series:~/Desktop$ appletviewer Main.html
```



Requesting Repainting: repaint() method

- Repaint method is responsible for handling update to paint cycle of applet.
- Whenever we want a component to repaint itself we need to call repaint method.
- In case we have made changes to appearance of a component but have not made any changes to its size than we can call repaint method to update new appearance of component on the graphical user interface.
- When call to repaint method is made, it performs request to erase and perform redraw of the component after a small delay in time. I.e it calls *update()* method to clear the screen and to display updated information by calling *paint()* method.
- Therefore both *repaint()* and *update()* methods call *paint()* to display the applet information.

Requesting Repainting: repaint() method

General formats of repaint() method:

void repaint()

void repaint(int left, int top, int width, int height)

void repaint(long max_delay)

void repaint(long max_delay, int left, int top, int width, int height)

Example: scrolltext.java